Darren Lee
danalee@ucsc.edu
CSE 13S Spring 2021
Assignment 3: Sorting: Putting your affairs in order

# Writeup

**Time Complexities and Constants**

The worst case and average case time complexity for Bubble Sort is $O(n^2)$. For small array sizes, Bubble Sort has a small enough constant that it performs reasonably well in comparison to Quick Sort and Shell Sort.

The worst case time complexity for Shell Sort using the Pratt Sequence is $O(n\log^2 n)$.

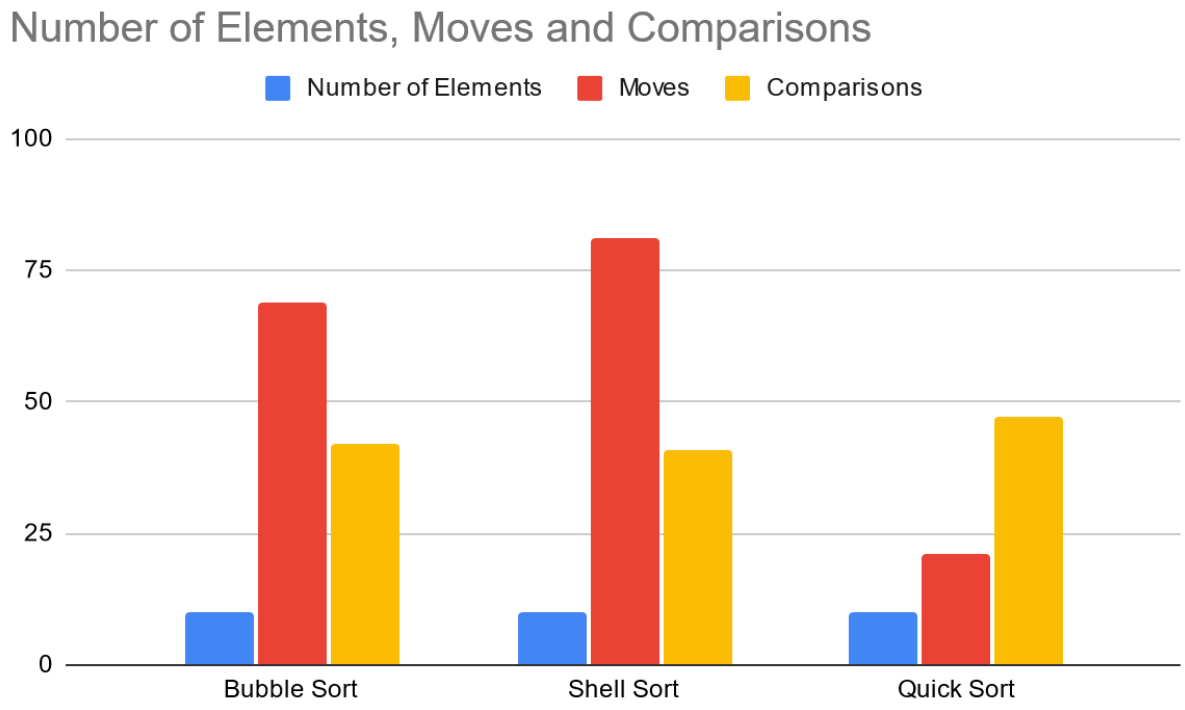
The worst case time complexity for Quick Sort is $O(n^2)$ just like Bubble Sort, but this case rarely happens and only occurs if the array is reverse-sorted and a poor pivot is chosen. Its average time complexity is O(nlogn), which is better than Bubble Sort and Shell Sort.

**What I Learned From the Sorting Algorithms**
I learned that comparison sorts are useful because we don't necessarily have to know the specifics of our data set or array of elements. Even without knowing the minimum or maximum of the array, comparison sorts allow us to create ordered arrays of elements just by checking whether elements are greater than or less than each other. I also learned that Quick Sort can be implemented recursively or iteratively if you make use of stacks or queues.

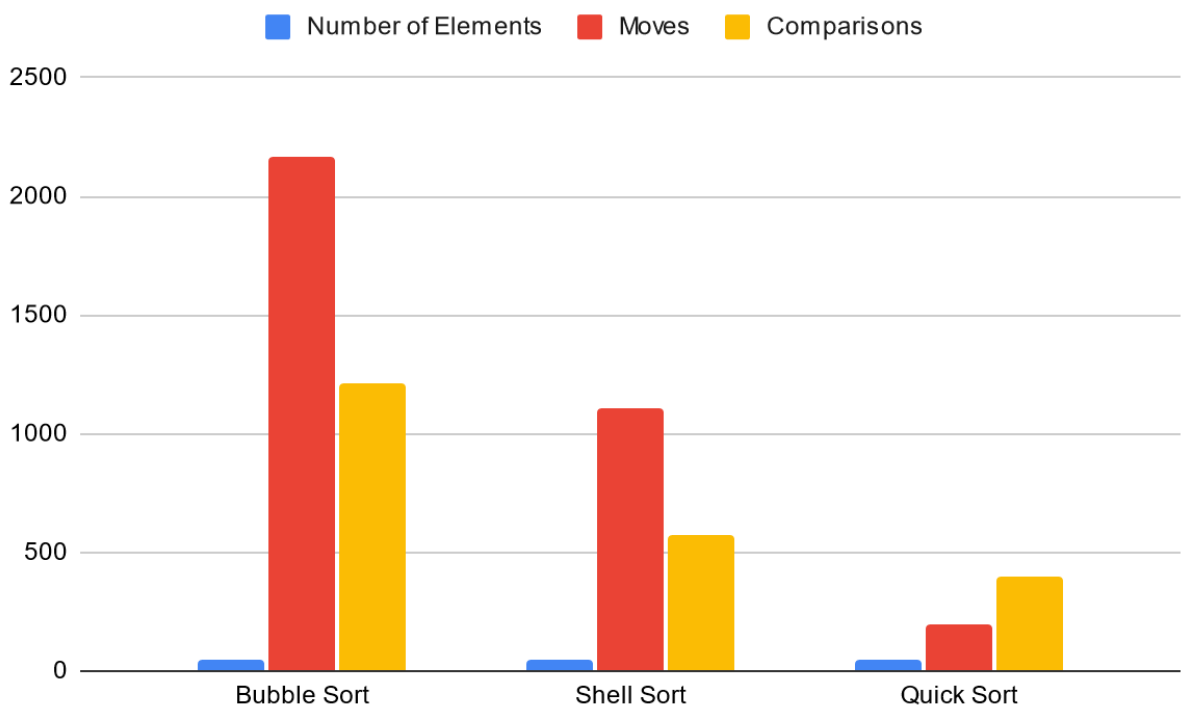
**Experimentation**
When testing out how fast the sorting algorithms were at larger values, I realized that Bubble Sort was not very usable for values above 100,000. Once the array size reaches such a large number, Bubble Sort makes over 700 million comparisons, which illustrates its $O(n^2)$ time complexity. Shell Sort is also quite slow for sizes in this range and makes over 8 million comparisons. Quick Sort, on the other hand, is very fast and manages to sort arrays of 100,000 in only around 2.4 million comparisons. When arrays are reverse-sorted, Bubble Sort and Quick Sort are both fairly fast. Shell Sort, on the other hand, runs about the same speed for a random array as a reverse-sorted array.

**Stack and Queue Size**
The stack never grows very large. Even for 1 million elements, it only grows to 64. Comparatively, the queue grows much faster and reaches around 160,000 for 1 million elements. Generally, the sizes of the stack and queue increase as the number of elements in the input array increases. At smaller values, the stack will double in size when the number of elements is multiplied by 10. The stack is multiplied by about 10 when the number of elements is multiplied by 10 for large numbers of input.

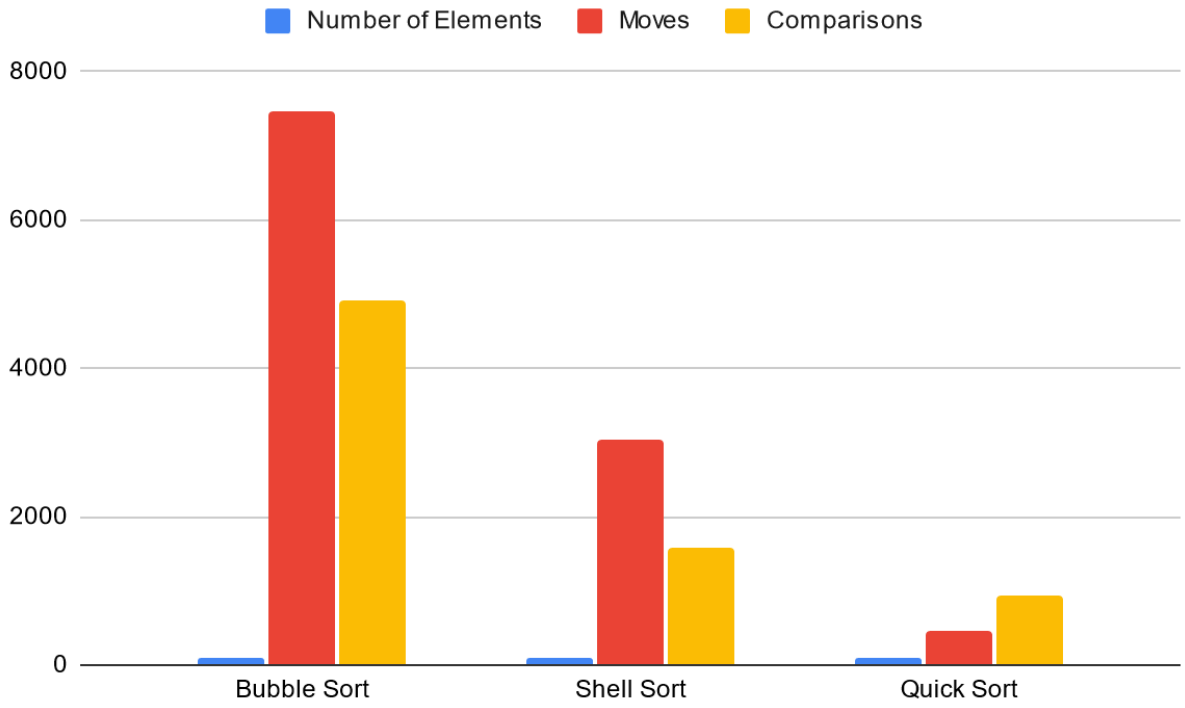
**Graphs**



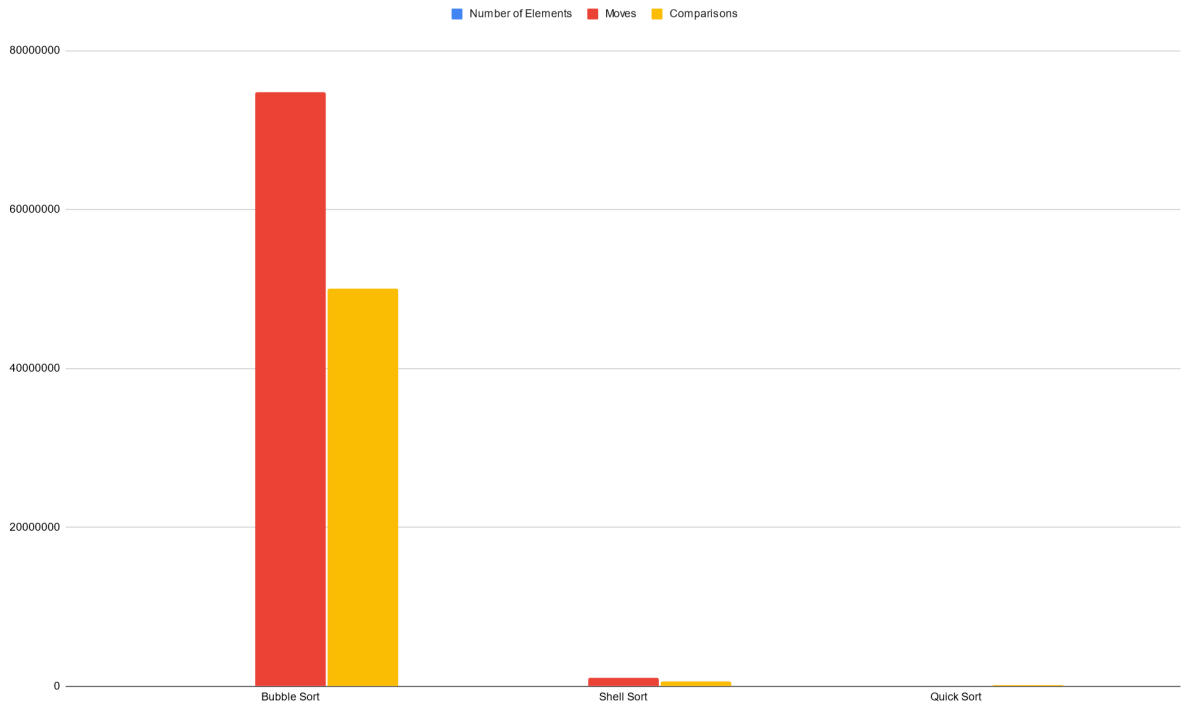

Array size=10




Array size=50




Array size=100

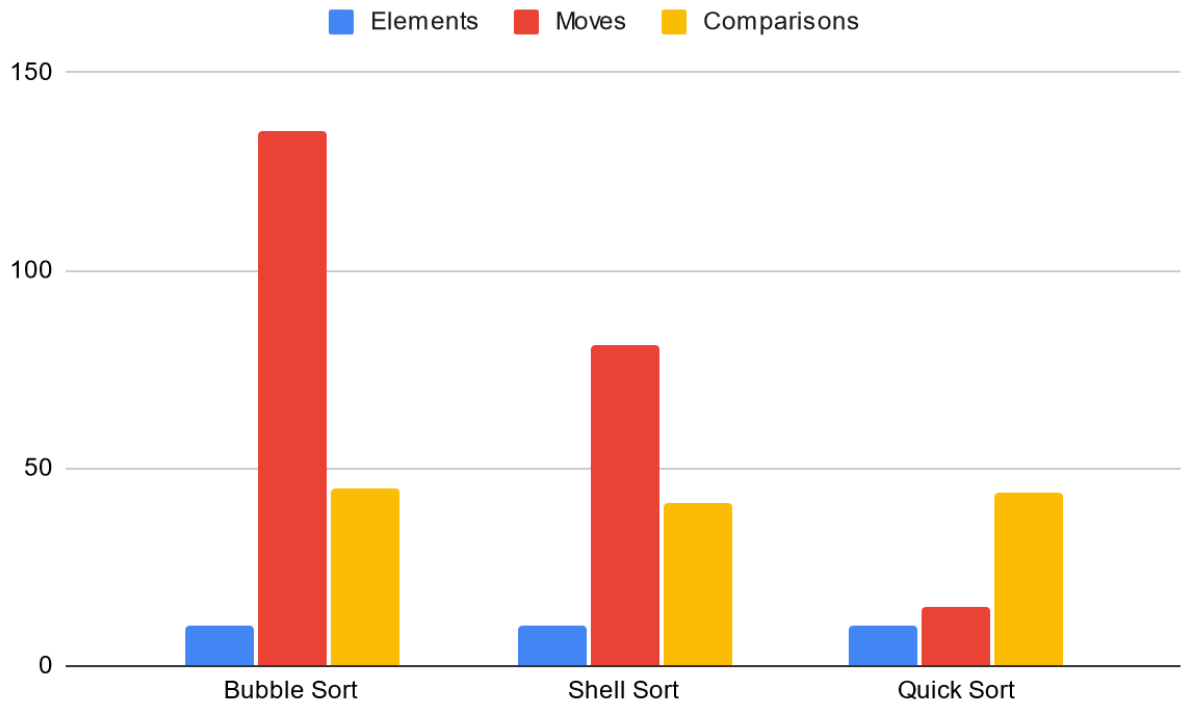



Array size=10,000

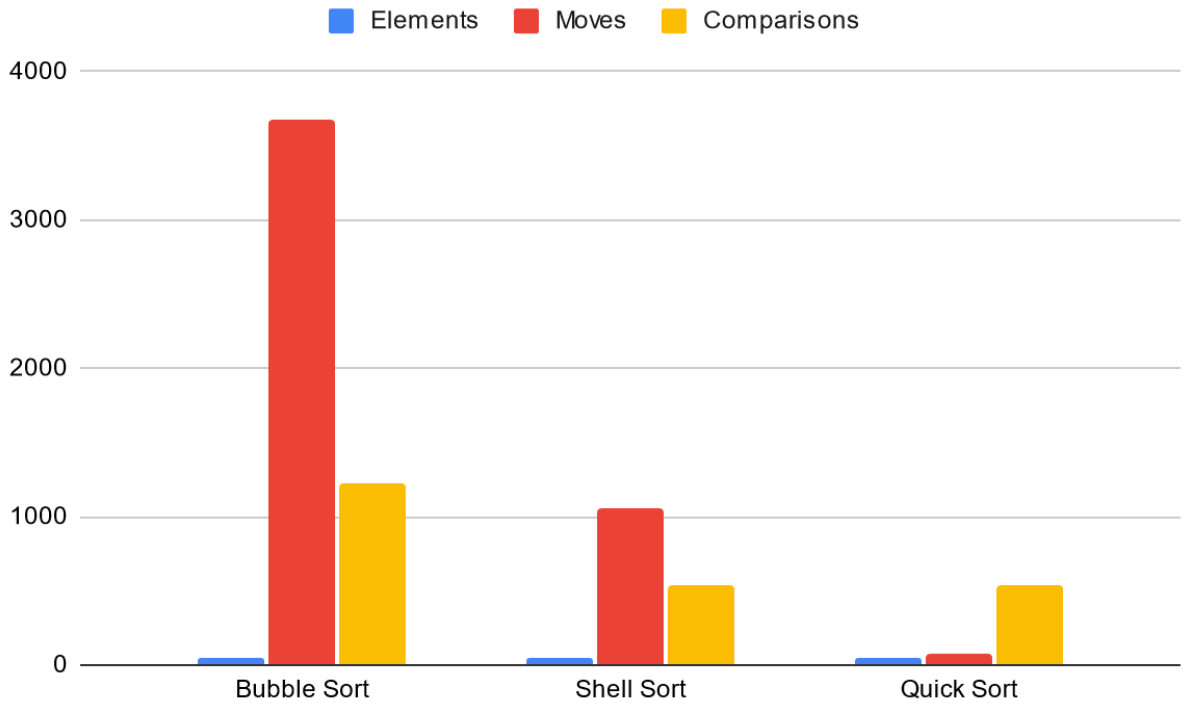
**Analysis**

For arrays with a small number of elements, the sorts are fairly similar. As the values grow larger, Bubble Sort becomes extremely slow, as does Shell Sort eventually. Quick Sort manages to maintain a relatively quick speed even for larger arrays. Bubble Sort and Shell Sort have worse time complexities on average than Quick Sort ($O(n^2)$ and $O(n\log^2 n)$ compared to $O(n\log n)$ respectively), which is why they are slower than Quick Sort as the array size n increases.

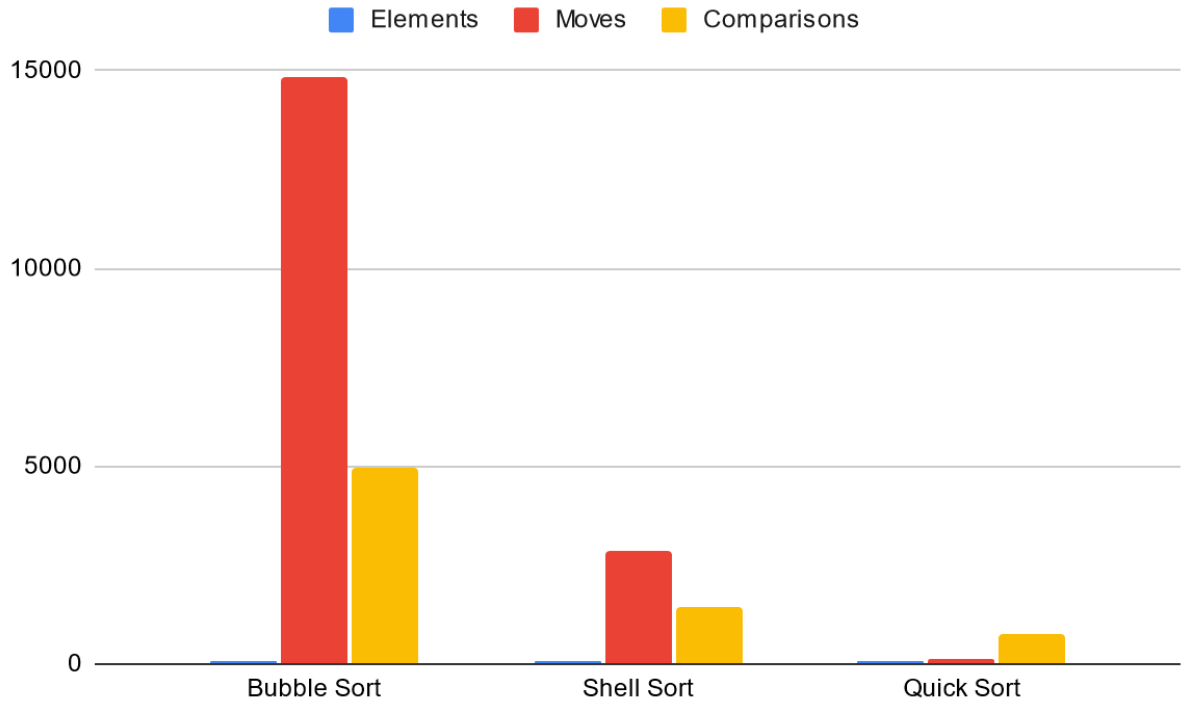
**Graphs for reverse-sorted arrays**



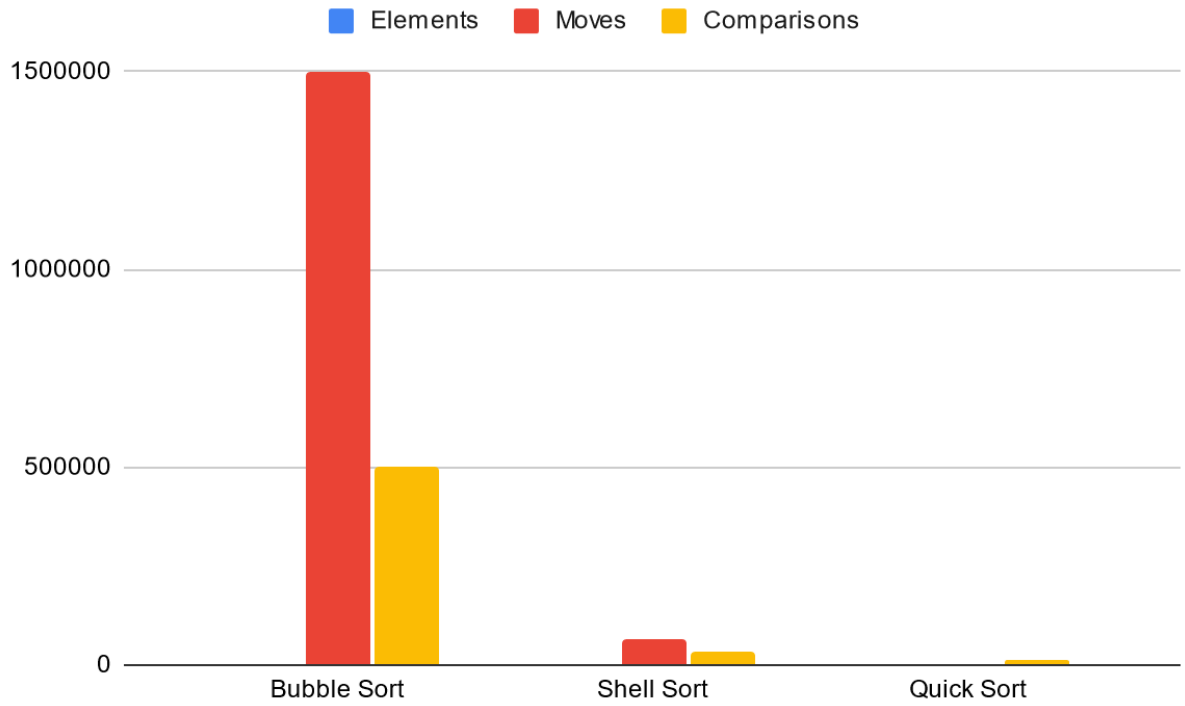

Reverse sorted, array size=10




Reverse sorted, array size=50




Reverse sorted, array size=100

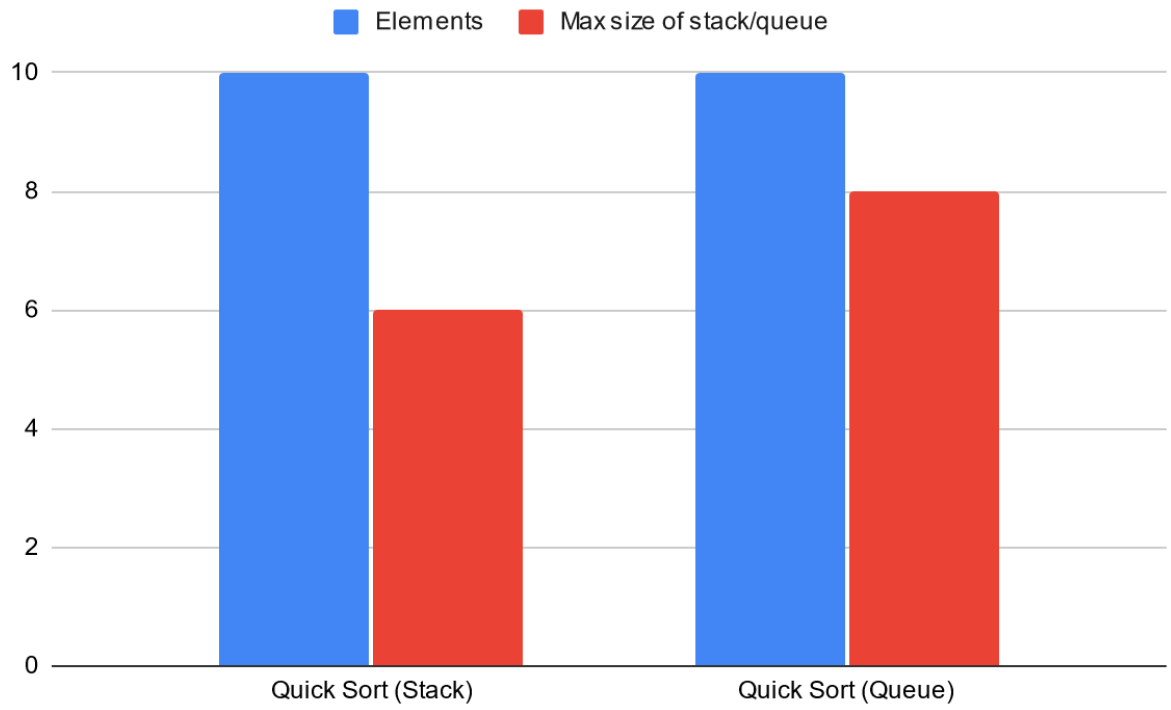



Reverse sorted, array size=1000

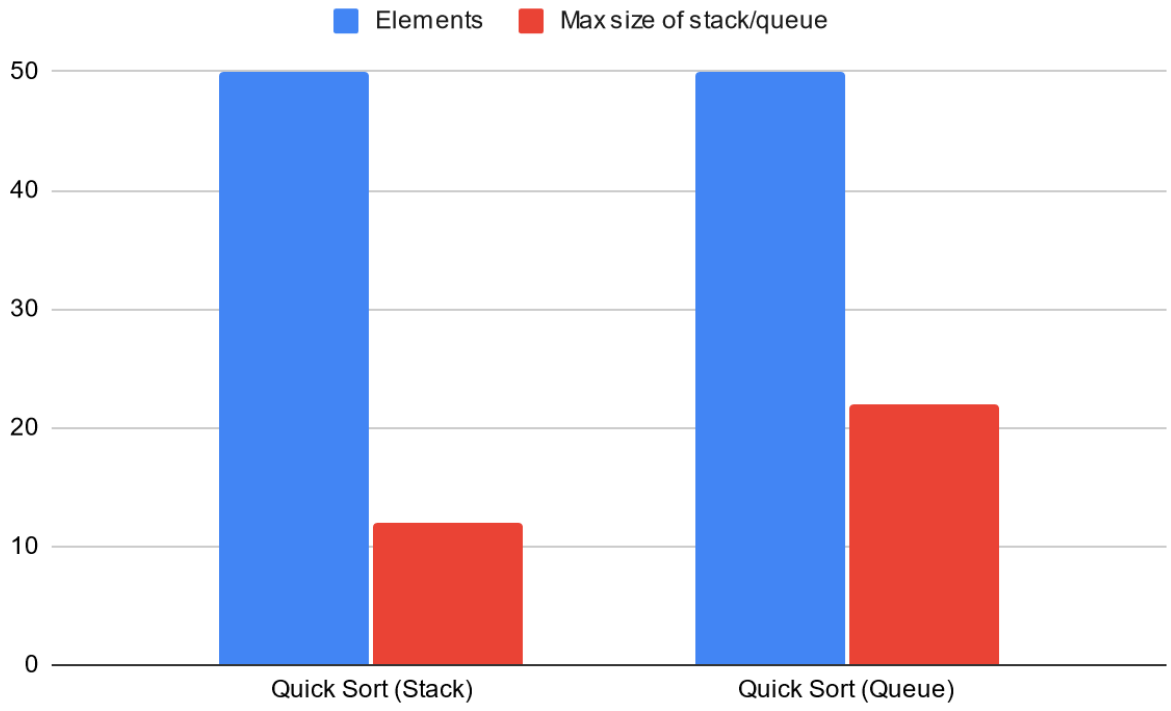
**Analysis**

Bubble Sort has the same time complexity of $O(n^2)$ for reverse-sorted arrays. However, it performs worse than it does with random arrays because it has to perform a swap for every pair of elements since they are all out of order. This is why a reverse-sorted array is the worst case for Bubble Sort. Shell Sort performs the same as it does with random arrays. This is due to the fact that Shell Sort only reverts to a Bubble Sort when the gap size is 1, but by that point Shell Sort has already sorted the array. Quick Sort is still fast with reverse-sorted arrays and uses very few moves and comparisons because as long as a good pivot is chosen, the subarrays will be divided evenly enough and there will not be a large amount of time spent sorting each smaller subarray.

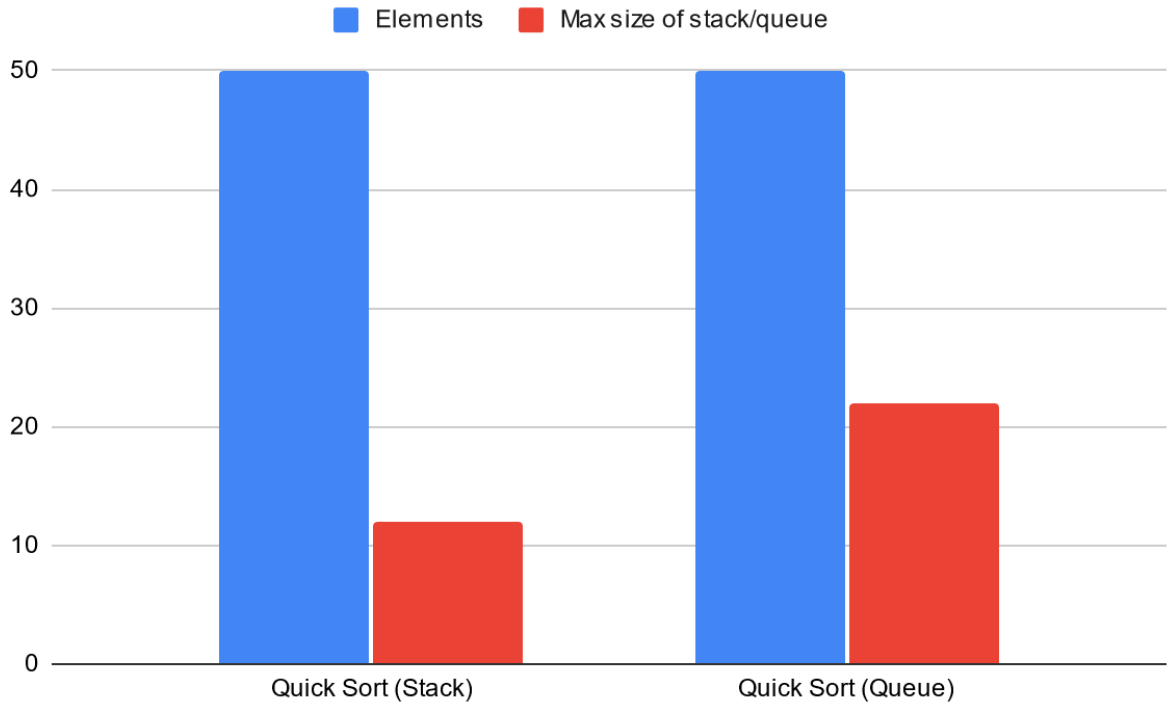
**Graphs for stack and queue sizes**



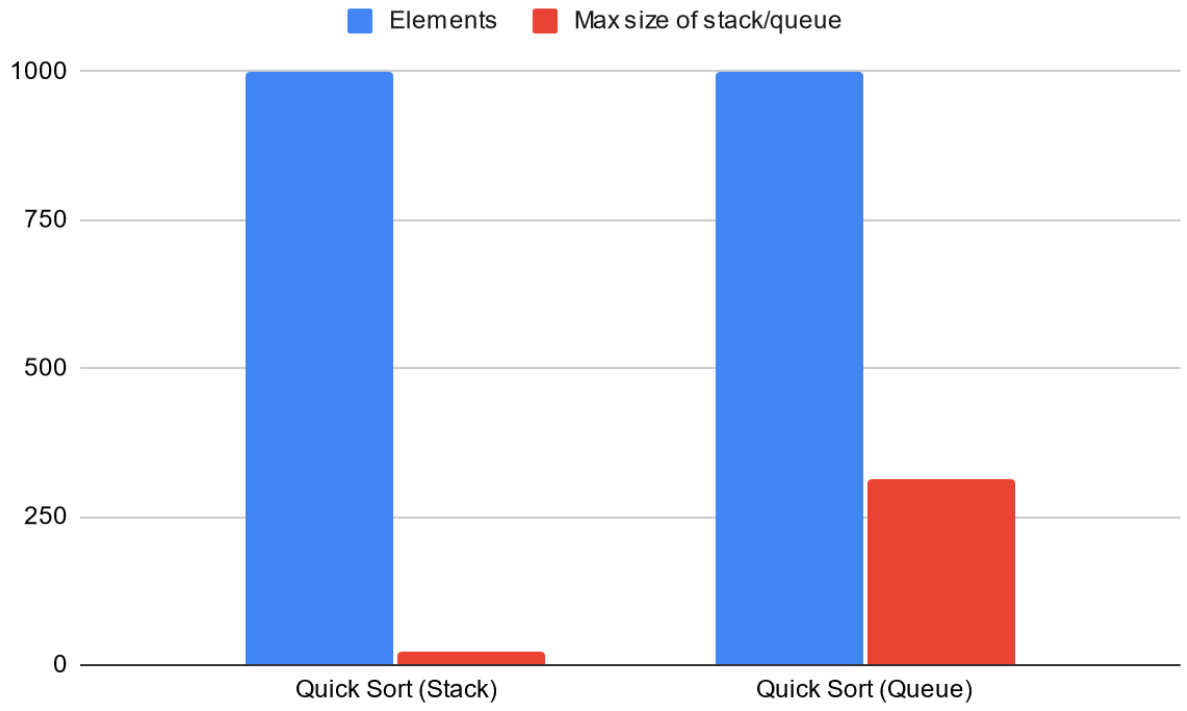

Array size=10



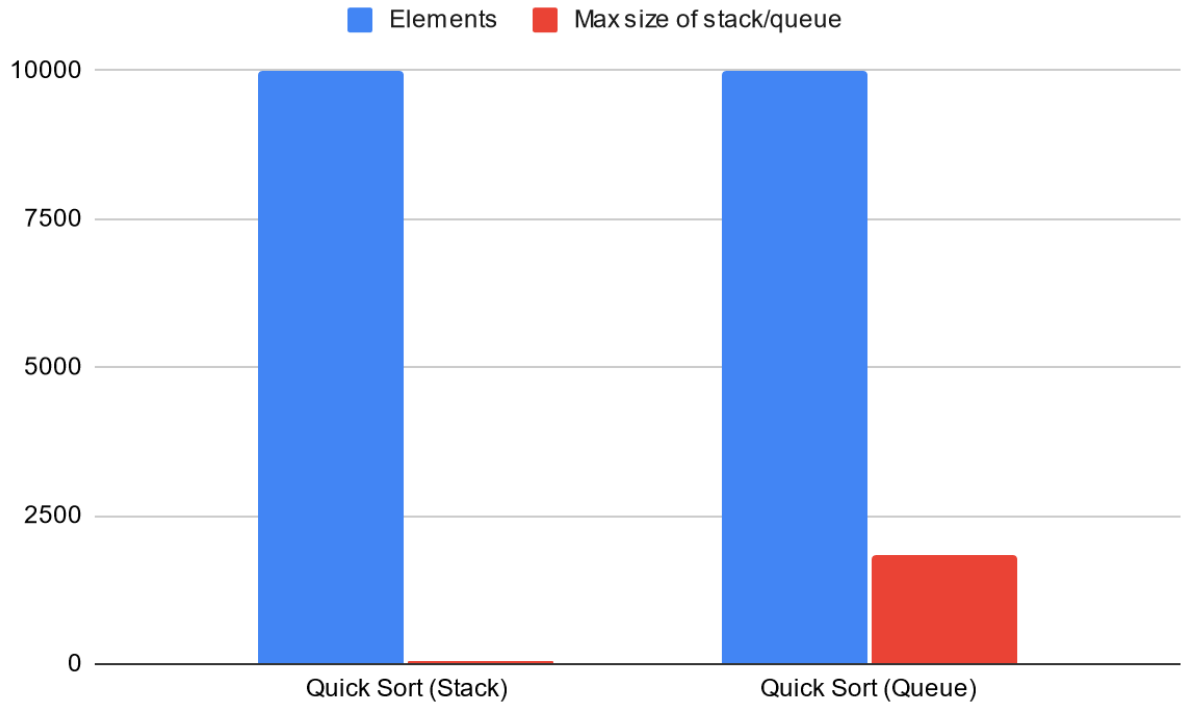

Array size=50




Array size=100




Array size=1000




Array size=10000

**Analysis**

In a Quick Sort the array is divided into subarrays, which will in turn keep being divided into smaller subarrays. The stack uses a last-in first-out (LIFO) policy and never grows very large even when the number of elements is large. The stack corresponds to a depth first search. With a depth first search, you can sort the smaller array first, and the space complexity will be logarithmic (O(logn)) since you push and pop the larger array repeatedly. The queue uses a first-in first-out (FIFO) policy and reaches a much larger maximum size than the queue for large arrays. This is because the queue corresponds to a breadth first search, and the smaller array is enqueued and dequeued repeatedly, leading to a space complexity of O(n).